Part II

3 Storage

When determining if some computing task is feasible it is often necessary to consider storage requirements. Let's take the following task as an example. You have been asked to run a simulation and afterwards produce a movie. Is this feasible? Obviously we don't have enough information to answer this question. Let's start with the following two questions.

1. How large is the simulation? Let's say 1000^3.
2. How long should the movie be? Let's say 40 seconds.

This is now enough information to answer the question, "how much storage do we need?", but maybe not directly. How is a movie produced? Quite simply, every frame of the movie is a render of one of the outputs of the simulation, so we need to produce an output for every frame. What we actually need to answer is:

1. How many outputs (movie frames) do we need?
2. What is the size of an output?
3. How many outputs do we need?

A movie is typically 25 frames per second[2], so how many frames (outputs) do we need for 40 seconds? That's simple: $25 \times 40 = 1000$ outputs. Next we need to know how large each output is? How many particles will be in each output? Again this is an easy question to answer. We have $1000^3 = 10^9$ (one billion) particles. Next we calculate how large each output will be. We will write Tipsy output files. How large will a single output be? We just multiple the number of particles times the size of each particle. A dark matter particle in Tipsy uses 36 bytes, so the size of each file is 36×10^9 or 36 GB. We need to store 1000 outputs, so we need 36 TB of storage.

Can you store this many outputs on your laptop? What is the size of the largest hard drive you can buy today? The answers are "no" (you don't have 36 TB of storage on your laptop), and "20 or so" TB. This problem has entered the realm of feasible, but annoying.

To process the data we also need to read these files into memory. Can you read 36 GB (one output) into the memory of your laptop? Probably not.

There are a number of approaches to handle these challenges.

- Use a large network storage system (with more than 36 TB of storage).
- Use a server to process the data (so we can read in 36 GB of data).
- Process the files in batches (so we need less disk storage).
- Read only the positions (and mass) into memory (reduces memory requirements to 16 GB).
- Stream process the data — read the particles one at a time and process them individual (is this even possible?)
- Build the frame (image) generation into the simulation code.

4 Multidimensional Arrays in C++

As previously discussed, the "C" and "C++" languages do not have native support for multidimensional arrays. Even the square bracket operator is not really even a one dimensional array, rather a convenient way of expressing pointer arithmetic.

[2] or 24 or 60 depending on the standard

4.1 Type promotion

What's wrong with this code?

```
int Nx = 2000;
int Ny = 2000;
int Nz = 2000;
int Ntotal = Nx * Ny * Nz;
```

Remember that `int` might be 32 bits which gives a range of $\pm 2^{31}$. This is less than 2000^3. We can use standard integers to "fix" this. What is wrong with this code.

```
#include <stdint.h>
...
uint32_t Nx = 2000;
uint32_t Ny = 2000;
uint32_t Nz = 2000;
uint64_t Ntotal = Nx * Ny * Nz;
```

The calculation are still done in `uint32_t` and only promoted at the very end. There are two ways to fix this.

```
#include <stdint.h>
...
uint64_t Nx = 2000;
uint64_t Ny = 2000;
uint64_t Nz = 2000;
uint64_t Ntotal = Nx * Ny * Nz;
```

Possibly better is to start with a `uint64_t` of 1 which causes all subsequent integer like values to be promoted to `uint64_t` before calculations are performed.

```
#include <stdint.h>
...
uint32_t Nx = 2000;
uint32_t Ny = 2000;
uint32_t Nz = 2000;
uint64_t Ntotal = (uint64_t)(1) * Nx * Ny * Nz;
```

In C++ it is better to use a **constructor** instead of a type cast as it is considered safer.

```
#include <cstdint>
...
std::uint32_t Nx = 2000;
std::uint32_t Ny = 2000;
std::uint32_t Nz = 2000;
std::uint64_t Ntotal = std::uint64_t(1) * Nx * Ny * Nz;
```

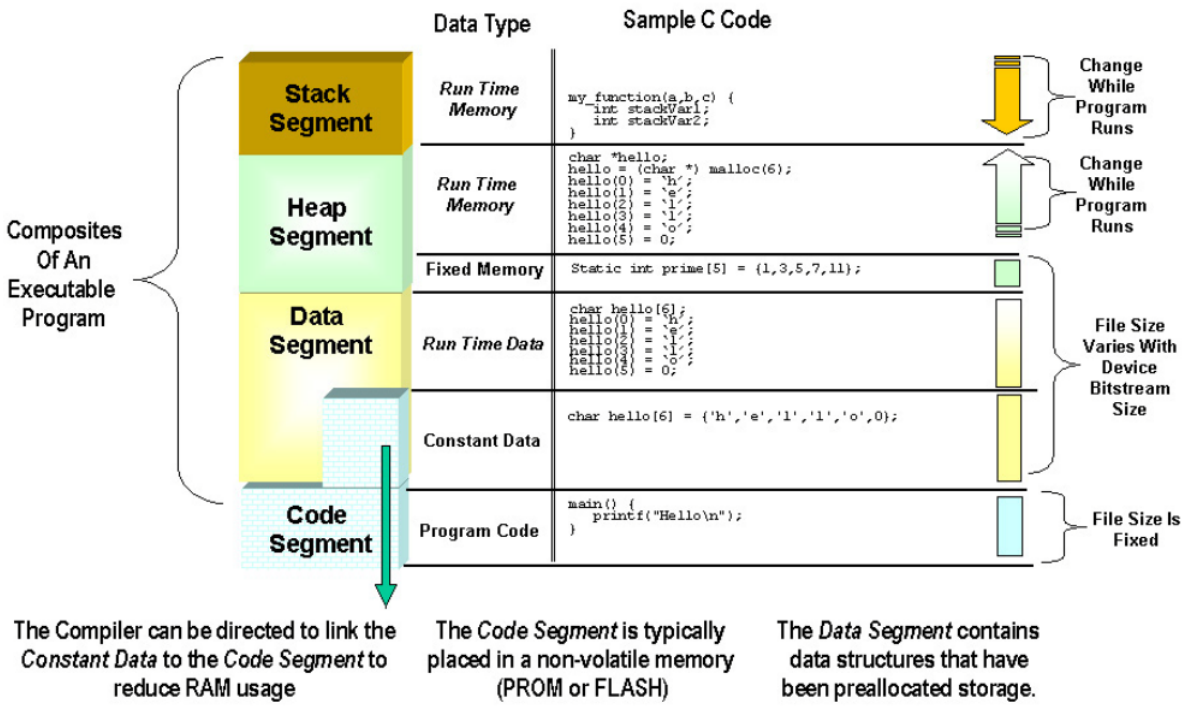
4.2 Stack and Heap

4.3 Allocation

Recall that memory in a computer is effectively one dimensional. All variables have an "address" and size and are located somewhere in the memory. The compiler and operating system cooperate to assign memory locations to your variables, sometime automatically, and sometimes when you explicitly ask for memory.

In classic C you use the `malloc` function to request storage for data, and you use the `free` function to release it when you are done. For example, the following code allocates space to store 10 floating point numbers; the size of a float is usually 4 bytes so it is requesting 40 bytes.

```
#include <stdlib.h>
...
float *pf = malloc(10 * sizeof(float));
...
free(pf);
```



The C++ language has first class support for allocating memory using `new` and `delete` to allocate a free memory. There is an important difference in C++ between allocating a single scalar, and allocating multiple values like we did above. Allocating a scaler is done in this way.

```
float *pf = new float;
...
delete pf;
```

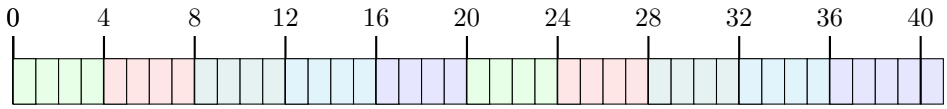
To allocate room for more than one element you must use the `[]` notation for `new`, but also for `delete` like so.

```
float *pf = new float[10];
...
delete [] pf;
```

If you allocate space for multiple values using `new[]` and subsequently release the memory with `delete` instead of `delete[]` then it is not correct and could cause your program to crash, or produce unexpected results.

4.4 Pointers

Above, we allocated room for 10 floating point numbers. In memory it would look like this.



In reality the starting address would not be zero, but rather some system assigned value like 12312400 or 928475834. We don't care what this actual value of the "address" is, only that there is a float stored there, another 4 bytes later and so on. We can now use our pointer to reference the data.

C	Value
`pf`	The address of the first float
`pf+1`	The address of the second float (4 bytes from the start)
`*pf`	The float at that address (the first float)
`*(pf+1)`	The second float
`pf[0]`	The first float
`pf[1]`	The second float
`pf[10]`	INVALID: references the non-existent eleventh float

There is no magic to pointers and pointer arithmetic. In fact, the following are the same.

$$p[i] \equiv *(p+i) \tag{3}$$

4.4.1 Strings are pointers and other weird things

In C and C++, a string is just a "pointer to char" (normally "const"):

```
#include <stdio.h>
int main() {
    const char *ps = "Hello world\n";
    printf("Hello world\n"); // prints Hello world
    printf(ps); // same thing
}
```

We can treat `char *` pointers like any other kind of pointer.

```
const char *ps = "Hello world\n";
char c1 = ps[0]; // c1 is is 'H'
char c2 = ps[6]; // c2 is is 'w'
char c3 = "is this valid"[5]; // Yes! c3 is 'i'
char c4 = 6["is this valid"]; // Yes! c4 is 's'
```

Even though `c4` looks weird it is completely valid because of equation 3. Because $p+i$ is the same as $i+p$ we can write it either way. Weird.

4.5 Memory Leaks

One of the classic problems with many programming languages is something called a memory leak. If you allocate memory, and forget to free it when you are done then that memory is lost and over time your program will consume more and more memory until it eventually crashes. Object oriented languages like C++ try to address this by abstracting away memory management into objects called **containers**. Earlier we learning about constructors — functions that are called when an object is created. The is also the concept of a destructor which is called when an object is deleted. C++ allows you to easily create objects that use memory that is automatically freed when you are finished.

4.6 Error Checking

There is a tendency to assume that functions operate successfully – open works, malloc is successful, and so on. Don't do this ever. It may seem like you are saving time, but you can easily spend hours debugging a problem that would have been obvious if you checked for errors. It's fine to simply abort there are problems if there is no sensible way to recover (obviously this is not always true and depends on the application and reliability requirements).

```
#include <stdio.h>  // fprintf, malloc
#include <stdlib.h> // abort

float * fp = malloc(10 * sizeof(float));
if (fp==nullptr) {
    fprintf(stderr,"Unable to allocate memory for data storage\n");
    abort();
}
```

4.7 1D Arrays

In Blitz++ we can easily create 1D arrays (or vectors) with the memory managed by an Array object.

```
#include "blitz/array.h"

void testfn(int a) {
    blitz::Array<float,1> data(10); // 1D float array with 10 element
    ...
    float f = data(1) + data(3); // Note we use parenthesis not brackets
}
```

This create a Blitz++ `Array` object with data elements of type `float` and `1` dimension. The constructor is called with `10` which creates a 10 element array. When the `testfn` function returns, the destructor is called and the memory is freed. Also note that we use parenthesis "`()`" instead of brackets "`[]`" for array indexing.

4.8 2D Arrays

We can easily create 2D arrays by changing the template parameter from a `1` to a `2`

```
#include "blitz/array.h"

void testfn(int a) {
    blitz::Array<float,2> data(5,5); // 2D float array with 5 x 5 element
    ...
    float f = data(0,2) + data(3,1); // Note we use parenthesis not brackets
}
```

4.9 Data Layout

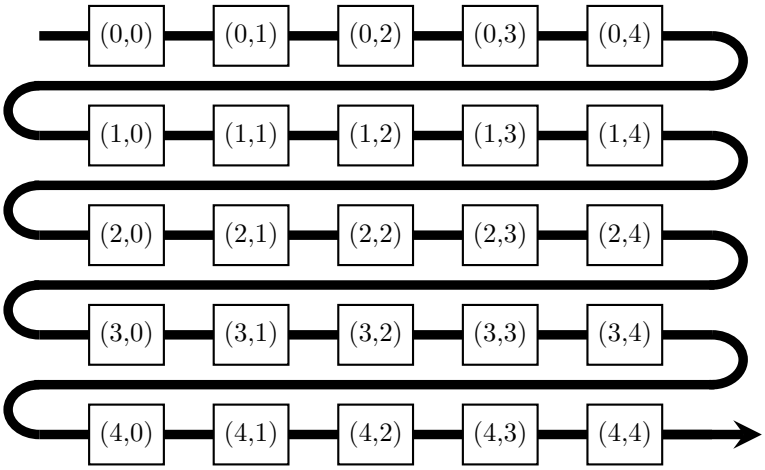
Already with two dimensions we have a choice to make in terms of how the data is organized in memory. Do we keep rows together (row major order), or do we keep columns together (column major order)? Why do we care? It actually makes a huge difference when we want to access the entire array; best is to run linearly through memory. So which loop is better?

```
void testfn(int a) {
    blitz::Array<float,2> data(5,5); // 2D float array with 5 x 5 element
    for(i=0; i<5; ++i) { // i is out loop
        for(j=0; j<5; ++j) { // j is inner loop
            sum += data(i,j)
        }
    }
}
```

```
void testfn(int a) {
    blitz::Array<float,2> data(5,5); // 2D float array with 5 x 5 element
    for(j=0; j<5; ++j) { // j is outer loop
        for(i=0; i<5; ++i) { // i is inner loop
            sum += data(i,j)
        }
    }
}
```

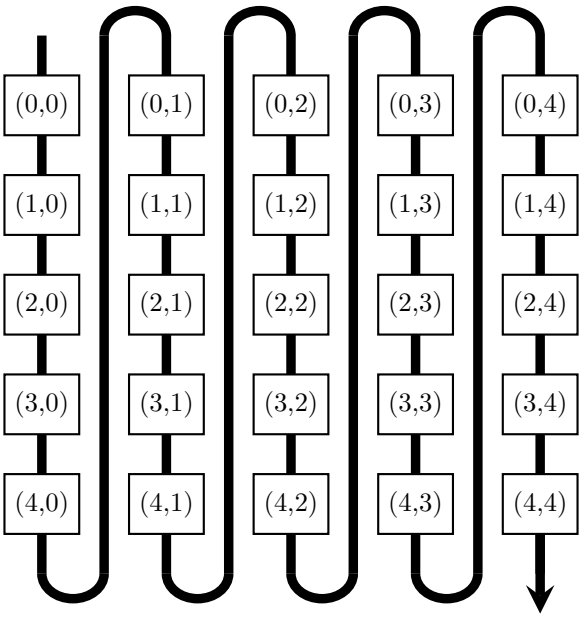
4.9.1 Row Major Order — "C" order

The classic C like layout is row major order. The first row is together in memory, followed by the second row and so on. The following diagram shows the layout. The arrow shows how each element is ordered in memory.



4.9.2 Column Major Order

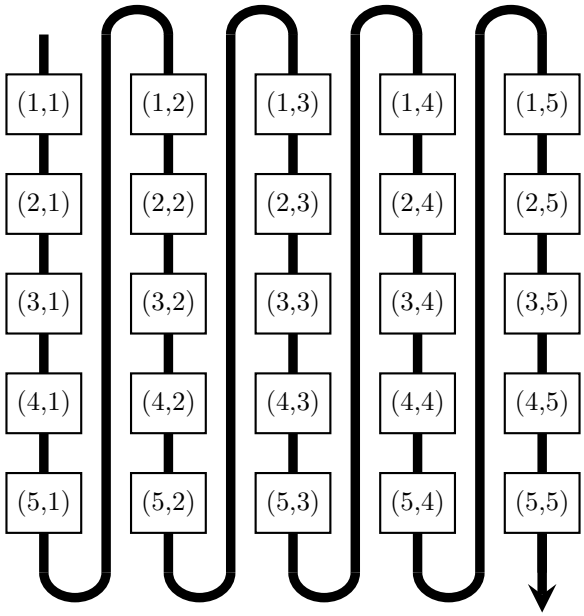
The alternative is column major order shown in the diagram below. Here elements in the same column are sequential in memory.



4.9.3 Column Major Order — "Fortran" order

In C and C++ the convention is to start indexes at zero and go to $n-1$. This is related to the way pointer arithmetic is performed. If the index is 0 and the data is pointed to by `p` then `p+0` points to the first element.

The situation is different with other languages such as `Fortran` where the index starts at one. The following diagram is identical to the previous one, except that indexes start at 1 and go to 5 instead of starting at 0 and going to 4.



4.9.4 Array Indexing

This brings us to the subject of array indexing. Recall that memory is inherently one dimensional, so any multi-dimensional indexing scheme will be translated to a one-dimensional scheme. In other words. We want to figure out how to map x and y to i in the following: `data(`x`,`y`)` $\longrightarrow$ `p[`i`]`.

We define the number of elements in each dimension as N_0 and N_1.

For C/C++ in row major order, the mapping of x and y to i is: $x + N_0 \times y$.

With Fortran we have: $N_1 \times (x-1) + (y-1)$.

More generally, we can define the array index base B_i and stride S_i like follows. Here we define a three dimensional array with indexes i_0, i_1 and i_2.

$$\begin{aligned} i &= \underbrace{1}_{\text{stride}} \times (i_0 - B_0) + \underbrace{N_0}_{\text{stride}} \times (i_1 - B_1) + \underbrace{N_0 \times N_1}_{\text{stride}} \times (i_2 - B_2) \\ &= \underbrace{S_0}_{\text{stride}} \times (i_0 - B_0) + \underbrace{S_1}_{\text{stride}} \times (i_1 - B_1) + \underbrace{S_2}_{\text{stride}} \times (i_2 - B_2) \end{aligned} \tag{4}$$

The array bases and strides are invariant so we can factor them out giving:

$$i = \underbrace{-S_0 \times B_0 - S_1 \times B_1 - S_2 \times B_2}_{\text{offset to 0,0,0}} + \underbrace{S_0 \times i_0 + S_1 \times i_1 + S_2 \times i_2}_{\text{index calculation}} \tag{5}$$

This is essentially what Blitz++ stores: a pointer to a (potentially fictitious) 0,0,0 coordinate, plus the strides for each dimension. By manipulating these values you can configure the array to be row-major or column major, and to start at any base you want (typically 0 or 1).

4.9.5 Data Layout in Blitz++

The Blitz++ library allows for complete flexibility of data layouts. Section 2.9 of the user guide describes this in detail. There are a few predefined layouts for common formats.

```
// 3D array in row major, base 0
blitz::Array<float,3> a1(3,4,5);
// 3D array in column major, base 0
blitz::Array<float,3> a2(3,4,5,blitz::ColumnMajorArray<3>());
// 3D array in column major, base 1
blitz::Array<float,3> a2(3,4,5,blitz::FortranArray<3>());
```

4.9.6 Iterating

Remember that we want to iterate through an array in memory order. One may be tempted to write the loops explicitly with knowledge of the order, and this does work but there is a better way: iterators!

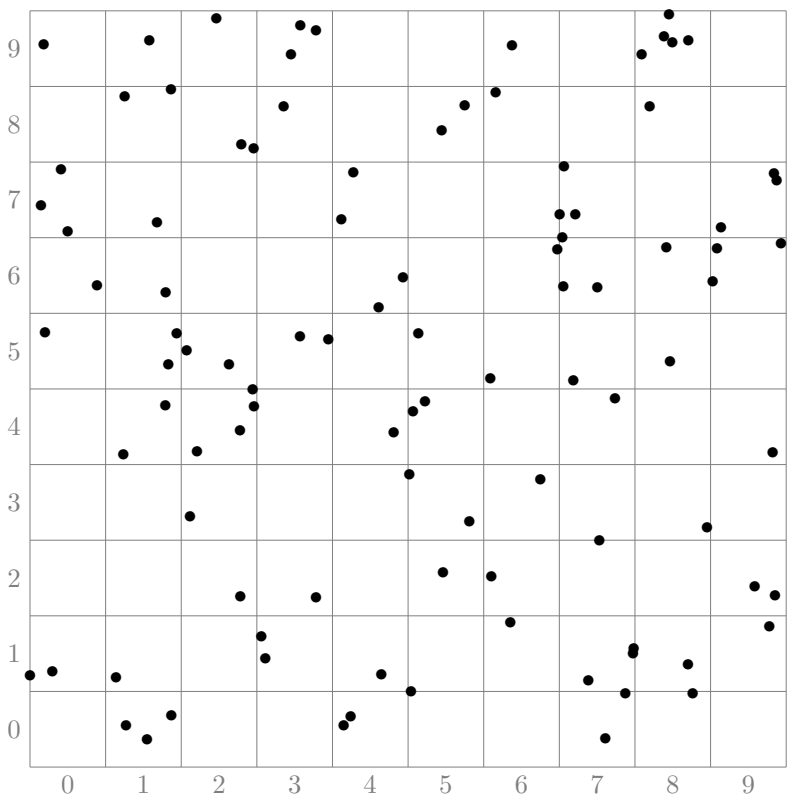
Instead of writing something like this:

```
int i,  j, k;
for(i=0; i<n0; ++i) {
    for(j=0;j<n1; ++j) {
        for(k=0; k<n2; ++k) {
            sum += data(i,j,k)
        }
    }
}
```

You can use iterators on the array. The iterator will loop over every element in order. Often you need to know the position in the array which you can get with position():

```
blitz::Array<float,3>::iterator it;
for(it=data.begin(); it!=data.end(); ++it) {
    int i = it.position()[0];
    int j = it.position()[1];
    int k = it.position()[2];
    sum += *it;
}
```

If you decide to change the data ordering, or write a function that take a Blitz++ array it can handle different data layouts optimally.

5 Mass Assignment (density estimator)

You have already seen Tipsy input files which contain a distribution of points in three dimensions.